

Toward Graph-Tokenizing Large Language Models with Reconstructive Graph Instruction Tuning

Zhongjian Zhang*
zhangzj@bupt.edu.cn
Beijing University of Posts and
Telecommunications
Beijing, China

Xiao Wang*
xiao_wang@buaa.edu.cn
Beihang University
Beijing, China

Mengmei Zhang
zhangmengmei@bestpay.com.cn
China Telecom Bestpay
Beijing, China

Jiarui Tan
tanjiarui@bupt.edu.cn
Beijing University of Posts and
Telecommunications
Beijing, China

Chuan Shi[†]
shichuan@bupt.edu.cn
Beijing University of Posts and
Telecommunications
Beijing, China

Abstract

The remarkable success of large language models (LLMs) has motivated researchers to adapt them as universal predictors for various graph-related tasks, with the ultimate goal of developing a graph foundation model that generalizes diverse scenarios. The key challenge is to align graph data with language spaces so that LLMs can better comprehend graphs. As a popular paradigm, Graph-Tokenizing LLMs (GTokenLLMs) encode complex structures and lengthy texts into a graph token sequence, and then align them with text tokens via language instructions tuning. Despite their initial success, our information-theoretic analysis reveals that existing GTokenLLMs rely solely on text supervision from language instructions, which achieve only implicit graph-text alignment, resulting in a text-dominant bias that underutilizes graph context. To overcome this limitation, we first prove that the alignment objective is upper-bounded by the mutual information between the input graphs and their hidden representations in the LLM, which motivates us to improve this upper bound to achieve better alignment. To this end, we further propose a reconstructive graph instruction tuning pipeline, **RGLM**¹. Our key idea is to reconstruct the graph information from the LLM's graph token outputs, explicitly incorporating graph supervision to constrain the alignment process. Technically, we embody RGLM by exploring three distinct variants from two complementary perspectives: RGLM-Decoder from the input space; RGLM-Similarizer and RGLM-Denoiser from the latent space. Additionally, we theoretically analyze the alignment effectiveness of each variant. Extensive experiments on various benchmarks and task scenarios validate the effectiveness of the proposed RGLM, paving the way for new directions in GTokenLLMs' alignment research.

*Both authors contributed equally to this research.

[†]Corresponding author.

¹The source code is available at <https://github.com/zhongjian-zhang/RGLM>



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792077>

CCS Concepts

• **Mathematics of computing** → *Graph algorithms*; • **Computing methodologies** → *Natural language generation*.

Keywords

Graph Foundation Models; Large Language Models; Graph Mining

ACM Reference Format:

Zhongjian Zhang, Xiao Wang, Mengmei Zhang, Jiarui Tan, and Chuan Shi. 2026. Toward Graph-Tokenizing Large Language Models with Reconstructive Graph Instruction Tuning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792077>

1 Introduction

Graph data are ubiquitous in the real world, such as social networks [16] and webpage networks [38]. Traditional graph neural networks (GNNs) [18, 40] are powerful tools for modeling graph structures, but they suffer from a weak multi-task handling capability [3]. Recently, the success of GPT-style LLMs [1, 6, 31] has motivated numerous studies to adapt them to the graph learning domain, particularly for text-attributed graphs (TAGs), where each node is associated with a text attribute. The ultimate goal is to build a graph foundation model that generalizes across diverse scenarios [23], in which LLMs are typically employed as predictors to accomplish various graph tasks, e.g., node classification, exhibiting promising potential and attracting considerable attention [5, 36, 51, 56].

A central challenge in this endeavor lies in aligning graph data with the natural language embedding space, so that LLMs can better comprehend graphs. To address this challenge, existing efforts mainly fall into two categories: Graph-Textualizing LLMs (GTextLLMs) [5, 8, 10, 35, 54] and Graph-Tokenizing LLMs (GTokenLLMs) [3, 20, 30, 34, 50]. The GTextLLMs utilize natural language to describe graphs via manually crafted prompt templates, directly aligning them with text. However, such strategies often suffer from redundancy and imprecise characterization [3, 5]. In contrast, GTokenLLMs construct various language instructions around graph tasks to perform tuning alignment, demonstrating superior performance while maintaining cost-effective training schemes. As shown

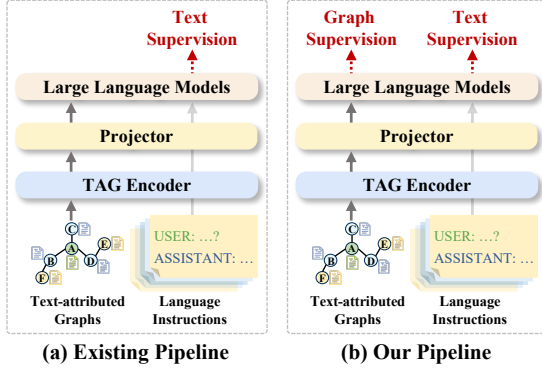


Figure 1: (a) Typical graph instruction tuning methods follow an LLM-centric design that solely leverages text supervision. (b) RGLM designs extra reconstructive graph tasks to explicitly incorporate graph supervision into the tuning process.

in Figure 1(a), GTokenLLMs typically adopt a TAG encoder to encode the complex structures and lengthy texts into a graph token sequence. This sequence is then aligned with the LLM embedding space via a projector, and subsequently used as prefix tokens for querying LLMs. However, this pipeline: $\text{TAG encoder} \rightarrow \text{projector} \rightarrow \text{LLM} \Leftarrow \text{language instructions}$ (where “ $\Leftarrow$ ” denotes supervision), is predominantly LLM-centric: (i) graph comprehension largely depends on graph-text alignment and the selected TAG encoder, and (ii) supervision derives exclusively from text data. This naturally raises a question: *Can language instructions alone effectively support graph-text alignment?* Answering this question is essential for advancing more effective and principled alignment strategies.

Here, we analyze the relationship between language instruction tuning and graph-text alignment from an information-theoretic perspective. In GTokenLLMs, the ideal optimization objective is to maximize the contribution of graph tokens to generating the current text token, given the preceding text context, which can be realized by maximizing the Conditional Graph-Text Mutual Information (C-GTMI). We prove that C-GTMI can be decomposed into two main terms (see Theorem 3.1): (i) a graph-text alignment term and (ii) a textual autoregressive dependency term. However, we find that the current GTokenLLMs [3, 30, 47] primarily optimize the second term, but neglect the explicit optimization of the first term, which achieves only implicit graph-text alignment. This leads to a bias toward textual information, potentially diminishing the significance of graph context in graph comprehension, yielding a suboptimal C-GTMI and limited alignment. Therefore, another question naturally arises: *How can we more effectively optimize C-GTMI to achieve better alignment?* Considering that LLMs’ semantic space is high-dimensional, designing a stable and effective optimization mechanism for this purpose remains a significant challenge.

In this paper, we first prove that the C-GTMI is upper-bounded by the mutual information between the input graph and its hidden representations in the LLM (see Lemma 3.1). This result motivates us to improve this upper bound to achieve better alignment, and this improvement can be realized through reconstruction. To this end, we further propose a novel alignment pipeline, RGLM, termed reconstructive graph instruction tuning. Our key idea is to reconstruct the graph information from the LLM’s graph token outputs, which are largely ignored in current GTokenLLMs. As illustrated in

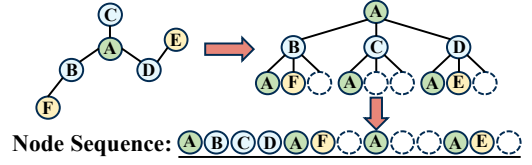


Figure 2: Neighbor detail template for graph serialization [3].

Figure 1(b), this pipeline explicitly incorporates graph supervision to constrain the alignment process. Technically, we embody RGLM by exploring three variants from two complementary perspectives: RGLM-Decoder, defined from the input space; RGLM-Similarizer and -Denoiser, defined from the latent space. Additionally, we theoretically analyze how each RGLM variant optimizes the upper bound of the C-GTMI. Our main contributions are as follows:

- We are the first to theoretically reveal that current GTokenLLMs using language instructions tuning achieve only implicit graph-text alignment, and prove that the alignment objective is upper-bounded by the mutual information between the input graphs and their hidden representations in the LLM.
- We propose a reconstructive graph instruction tuning pipeline, called RGLM, and explore three distinct variants: RGLM-Decoder from the input space, RGLM-Similarizer and RGLM-Denoiser from the latent spaces. Additionally, we theoretically analyze the alignment effectiveness of each variant.
- Experimental results on four datasets demonstrate that the proposed RGLM, which introduces different reconstructive graph losses, achieves substantial improvements in supervised learning, multi-dataset generalization, and zero-shot capability, highlighting its potential as a graph foundation model.

2 Preliminary

2.1 Text-attributed Graphs (TAGs)

We define a TAG as $\mathcal{G}_{\text{full}} = (\mathcal{V}_{\text{full}}, \mathcal{E}_{\text{full}}, \mathcal{S}_{\text{full}})$, where $\mathcal{V}_{\text{full}} = \{v_i\}_{i=1}^{|\mathcal{V}_{\text{full}}|}$, $\mathcal{E}_{\text{full}} = \{e_i\}_{i=1}^{|\mathcal{E}_{\text{full}}|}$, $\mathcal{S}_{\text{full}} = \{s_i\}_{i=1}^{|\mathcal{S}_{\text{full}}|}$ denote the node, edge, and text set, respectively. The text set $\mathcal{S}_{\text{full}}$ is typically encoded into node features $\mathbf{Z}_{\text{full}} = \{z_1, \dots, z_{|\mathcal{V}_{\text{full}}|}\}$ using pre-trained language models [7, 28]. To efficiently process node-level tasks with LLMs, e.g., node classification, it is common to sample a subgraph centered at a node to reduce computational cost and irrelevant information [3, 30]. Formally, we denote $\mathcal{G}_{\text{sub}}^h(v) = (\mathcal{V}_v^h, \mathcal{E}_v^h, \mathbf{Z}_v^h)$ as the h -hop subgraph centered at node v , where $\mathcal{V}_v^h$ includes all nodes within h hops of v , $\mathcal{E}_v^h$ contains their interconnecting edges, and $\mathbf{Z}_v^h$ is the corresponding feature matrix. For simplicity, we use $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathbf{Z}\}$ to denote such subgraphs when clear from context.

2.2 Graph-Tokenizing LLMs (GTokenLLMs)

Generally, a θ -parameterized LLM models the canonical causal distribution over a text token sequence $\mathbf{x}_{1:T} = (x_1, \dots, x_T)$ of length T as $p_{\theta}(\mathbf{x}) = \prod_{i=1}^T p_{\theta}(x_i | \mathbf{x}_{<i})$. To enable LLMs to understand graph content, GTokenLLMs encode complex structures and lengthy texts into a graph token sequence, and treat them as prefix tokens for querying LLMs. As illustrated in Figure 3(a), a graph $\mathcal{G}$ is first encoded as a token sequence via a TAG encoder $\mathcal{F}$, where $\mathcal{F}$ can be a predefined GNN [12, 20, 50] or a graph serialization template [3, 30, 47]. These tokens are subsequently projected into

the LLM’s input space via a ϕ -parameterized projector $\mathcal{P}$. Thus, the canonical causal distribution is reformulated as:

$$p_{\Theta}(\mathbf{x}) = \prod_{i=1}^T p_{\Theta}(\mathbf{x}_i | \mathbf{v}, \mathbf{x}_{<i}), \quad \mathbf{v} = \mathcal{P}_{\phi} \circ \mathcal{F}(\mathcal{G}), \quad (1)$$

where $\Theta = \{\theta, \phi\}$ is the parameters, $\mathbf{v}_{1:N} = (\mathbf{v}_1, \dots, \mathbf{v}_N)$ is the graph token sequence of length N . In this paper, we adopt the state-of-the-art graph serialization Neighbor Detail Template [3] as $\mathcal{F}$, owing to its elaborate descriptions of a node and its surrounding neighborhood, which is more suitable for our subsequent reconstruction task. As illustrated in Figure 2, given a central node A , we construct a fixed-size computational tree rooted at A to represent its h -hop subgraph. In this tree, A serves as the root, its 1-hop neighbors are assigned as its children, 2-hop neighbors as the children of 1-hop neighbors, and so on. When a node has more neighbors than the predefined size, we randomly sample a subset; if it has fewer neighbors, we insert placeholder nodes to maintain structural consistency. The tree is then traversed in level-order to produce a fixed-length graph token sequence, where each token corresponds to a node and is represented by its feature $\mathbf{z}$. This process effectively transforms the graph structure into a sequence while preserving the hierarchical neighborhood information.

Typically, GTokenLLMs are trained with supervised instruction tuning, where the model parameters Θ are optimized by minimizing an autoregressive loss on language supervision (e.g., textual labels of node classification). In this setting, the autoregressive loss enforces next-token prediction, i.e., each input token yields one output conditioned on the preceding context. The overall objective is to maximize the log-likelihood of the supervised textual outputs.

$$\mathcal{L}_{\text{text}}(\Theta = \{\theta, \phi\}, \mathcal{G}, \mathbf{x}) = \frac{-1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \log p_{\Theta}(\mathbf{x}_i | \mathbf{v}, \mathbf{x}_{<i}), \quad (2)$$

where $\mathcal{S} \subseteq \{N+1, \dots, T\}$ denotes the index set of textual label tokens in the input sequence. As indicated in Equation (2), only the text outputs $\mathbf{x}_{i \in \mathcal{S}}$ are supervised.

3 Motivation Analysis

3.1 Implicit Graph-Text Alignment of Existing GTokenLLMs

Given an input token sequence of length T , consisting of a prefix graph token sequence $\mathbf{v}_{1:N} = (\mathbf{x}_1, \dots, \mathbf{x}_N)$ and a text token sequence $\mathbf{x}_{1:M} = (\mathbf{x}_1, \dots, \mathbf{x}_M)$, where $M = T - N$. At decoding step t , GTokenLLMs predict $\mathbf{x}_t$ conditioned on $\mathbf{v}$ and $\mathbf{x}_{<t}$. We denote entropy by $\mathcal{H}(\cdot)$ and mutual information by $\mathcal{I}(\cdot; \cdot)$. The ideal optimization objective of GTokenLLMs is to maximize the contribution of graph tokens to generating the current text token, given the preceding text context. To formalize it, we first give the definition:

DEFINITION 3.1 (CONDITIONAL GRAPH-TEXT MUTUAL INFORMATION (C-GTMI)). *At decoding step t , let $\mathbf{s}^G$ and $\mathbf{s}_{<t}^T$ denote the LLM’s hidden representation of the graph tokens and the preceding text tokens, respectively. The C-GTMI is defined as $\mathcal{I}(\mathbf{x}_t; \mathbf{s}^G | \mathbf{s}_{<t}^T)$, which measures how much the graph tokens contribute to generating the current text token given the preceding text context.*

Based on the above definition, the optimization objective of GTokenLLMs can be realized by maximizing the C-GTMI.

THEOREM 3.1 (IMPLICIT GRAPH-TEXT ALIGNMENT VIA AUTOREGRESSIVE LOSS). *The C-GTMI can be decomposed as:*

$$\mathcal{I}(\mathbf{x}_t; \mathbf{s}^G | \mathbf{s}_{<t}^T) = \mathcal{I}(\mathbf{x}_t; \mathbf{s}^G) + \mathcal{I}(\mathbf{x}_t; \mathbf{s}_{<t}^T | \mathbf{s}^G) - \mathcal{I}(\mathbf{x}_t; \mathbf{s}_{<t}^T), \quad (3)$$

where $\mathcal{I}(\mathbf{x}_t; \mathbf{s}^G)$ reflects graph-text alignment, $\mathcal{I}(\mathbf{x}_t; \mathbf{s}_{<t}^T | \mathbf{s}^G)$ captures textual autoregressive dependencies, and $\mathcal{I}(\mathbf{x}_t; \mathbf{s}_{<t}^T)$ depends only on the textual data distribution.

Theorem 3.1 reveals that the autoregressive loss on language supervision primarily optimizes the textual autoregressive dependency, but only implicitly encourages graph-text alignment, namely that $\mathcal{I}(\mathbf{x}_t; \mathbf{s}^G)$ is not explicitly optimized. Consequently, the model tends to overly rely on textual information, potentially diminishing the significance of graph context in graph comprehension, yielding a suboptimal C-GTMI and limited performance.

PROOF. In practice, the autoregressive training objective of GTokenLLMs is the cross-entropy loss between the true distribution p and the model distribution q_{θ} . Equivalently, at step t , this can be written as the negative log-likelihood:

$$\begin{aligned} \mathcal{L}_{\text{CE}} &= -\mathbb{E}[\log q_{\theta}(\mathbf{x}_t | \mathbf{v}, \mathbf{x}_{<t})] \\ &= \mathcal{H}(\mathbf{x}_t | \mathbf{v}, \mathbf{x}_{<t}) + \mathbb{E}[\text{KL}p(\mathbf{x}_t | \cdot) \parallel q_{\theta}(\mathbf{x}_t | \cdot)] \\ &\geq \mathcal{H}(\mathbf{x}_t | \mathbf{v}, \mathbf{x}_{<t}). \end{aligned} \quad (4)$$

Since decoder-only LLMs generate tokens based on their internal hidden states rather than directly on raw input tokens, the conditional probability of the next text token can be equivalently expressed in terms of these hidden representations. Thus, we have:

$$q_{\theta}(\mathbf{x}_t | \mathbf{v}, \mathbf{x}_{<t}) = q_{\theta}(\mathbf{x}_t | \mathbf{s}^G, \mathbf{s}_{<t}^T). \quad (5)$$

This formulation highlights that the prediction of the next token depends solely on the internal hidden representations of both modalities, providing a foundation for analyzing alignment at the level of hidden states. Substituting this into Equation (4) yields:

$$\mathcal{L}_{\text{CE}} = -\mathbb{E}[\log q_{\theta}(\mathbf{x}_t | \mathbf{s}^G, \mathbf{s}_{<t}^T)] \geq \mathcal{H}(\mathbf{x}_t | \mathbf{s}^G, \mathbf{s}_{<t}^T). \quad (6)$$

According to the mutual information identity:

$$\mathcal{H}(\mathbf{x}_t | \mathbf{s}^G, \mathbf{s}_{<t}^T) = \mathcal{H}(\mathbf{x}_t | \mathbf{s}_{<t}^T) - \mathcal{I}(\mathbf{x}_t; \mathbf{s}^G | \mathbf{s}_{<t}^T). \quad (7)$$

Intuitively, $\mathcal{H}(\mathbf{x}_t | \mathbf{s}_{<t}^T)$ is determined by the data distribution, and minimizing $\mathcal{H}(\mathbf{x}_t | \mathbf{s}^G, \mathbf{s}_{<t}^T)$ is equivalent to maximizing the C-GTMI $\mathcal{I}(\mathbf{x}_t; \mathbf{s}^G | \mathbf{s}_{<t}^T)$. By the chain rule of mutual information, we have:

$$\mathcal{I}(\mathbf{x}_t; \mathbf{s}^G | \mathbf{s}_{<t}^T) = \mathcal{I}(\mathbf{x}_t; \mathbf{s}^G) + \mathcal{I}(\mathbf{x}_t; \mathbf{s}_{<t}^T | \mathbf{s}^G) - \mathcal{I}(\mathbf{x}_t; \mathbf{s}_{<t}^T). \quad (8)$$

□

3.2 Towards Explicit Graph-Text Alignment

To achieve better alignment by more effectively optimizing C-GTMI, we first theoretically establish an upper bound for C-GTMI:

LEMMA 3.1 (UPPER BOUND OF C-GTMI). *Given an input graph $\mathcal{G}$, the C-GTMI is upper-bounded by the mutual information between the graph $\mathcal{G}$ and its hidden representations $\mathbf{s}^G$ in the LLM, i.e.,*

$$\mathcal{I}(\mathbf{x}_t; \mathbf{s}^G | \mathbf{s}_{<t}^T) \leq \mathcal{I}(\mathcal{G}; \mathbf{s}^G). \quad (9)$$

Lemma 3.1 motivates us to optimize the lower bound of $\mathcal{I}(\mathcal{G}; \mathbf{s}^G)$, and this improvement can be further realized through reconstruction [4, 13], which in turn improves the attainable upper bound of

C-GTMI. This not only explicitly optimizes the graph-text alignment term $I(\mathbf{x}_t; \mathbf{s}^G)$, but also enhances autoregressive dependency modeling $I(\mathbf{x}_t; \mathbf{s}_{<t}^T | \mathbf{s}^G)$, thereby achieving better alignment.

PROOF. Based on the chain rule of mutual information, we first analyze the $I(\mathcal{G}, \mathbf{s}^G, \mathbf{x}_t)$ under the condition $\mathbf{s}_{<t}^T$ and have:

$$\begin{aligned} I(\mathcal{G}; \mathbf{s}^G, \mathbf{x}_t | \mathbf{s}_{<t}^T) &= I(\mathcal{G}; \mathbf{s}^G | \mathbf{s}_{<t}^T) + I(\mathcal{G}; \mathbf{x}_t | \mathbf{s}^G, \mathbf{s}_{<t}^T) \\ &= I(\mathcal{G}; \mathbf{x}_t | \mathbf{s}_{<t}^T) + I(\mathcal{G}; \mathbf{s}^G | \mathbf{x}_t, \mathbf{s}_{<t}^T). \end{aligned} \quad (10)$$

By equating the two expressions and rearranging terms, and then applying the symmetry of mutual information, we obtain:

$$\begin{aligned} I(\mathcal{G}; \mathbf{s}^G | \mathbf{s}_{<t}^T) - I(\mathcal{G}; \mathbf{s}^G | \mathbf{x}_t, \mathbf{s}_{<t}^T) \\ &= I(\mathcal{G}; \mathbf{x}_t | \mathbf{s}_{<t}^T) - I(\mathcal{G}; \mathbf{x}_t | \mathbf{s}^G, \mathbf{s}_{<t}^T) \\ &= I(\mathbf{x}_t; \mathcal{G} | \mathbf{s}_{<t}^T) - I(\mathbf{x}_t; \mathcal{G} | \mathbf{s}^G, \mathbf{s}_{<t}^T) \\ &= I(\mathbf{x}_t; \mathbf{s}^G | \mathbf{s}_{<t}^T) - I(\mathbf{x}_t; \mathbf{s}^G | \mathcal{G}, \mathbf{s}_{<t}^T). \end{aligned} \quad (11)$$

Given that graph tokens $\mathbf{v}$ are typically input as prefix tokens into decoder-only LLMs, the $\mathbf{s}^G$ are deterministically derived from $\mathcal{G}$, thus we have $I(\mathbf{x}_t; \mathbf{s}^G | \mathcal{G}, \mathbf{s}_{<t}^T) = 0$, yielding the identity:

$$\begin{aligned} I(\mathbf{x}_t; \mathbf{s}^G | \mathbf{s}_{<t}^T) &= I(\mathcal{G}; \mathbf{s}^G | \mathbf{s}_{<t}^T) - I(\mathcal{G}; \mathbf{s}^G | \mathbf{x}_t, \mathbf{s}_{<t}^T) \\ &= \mathcal{H}(\mathbf{s}^G | \mathbf{s}_{<t}^T) - I(\mathcal{G}; \mathbf{s}^G | \mathbf{x}_t, \mathbf{s}_{<t}^T) \\ &\leq \mathcal{H}(\mathbf{s}^G | \mathbf{s}_{<t}^T). \end{aligned} \quad (12)$$

Since $I(\mathbf{s}^G; \mathbf{s}_{<t}^T) = \mathcal{H}(\mathbf{s}^G) - \mathcal{H}(\mathbf{s}^G | \mathbf{s}_{<t}^T) \geq 0$, it follows:

$$\mathcal{H}(\mathbf{s}^G | \mathbf{s}_{<t}^T) \leq \mathcal{H}(\mathbf{s}^G). \quad (13)$$

Moreover, since $\mathbf{s}^G$ is a deterministic function of $\mathcal{G}$:

$$I(\mathcal{G}; \mathbf{s}^G) = \mathcal{H}(\mathbf{s}^G) - \mathcal{H}(\mathbf{s}^G | \mathcal{G}) = \mathcal{H}(\mathbf{s}^G). \quad (14)$$

Combining the inequalities yields the final result:

$$I(\mathbf{x}_t; \mathbf{s}^G | \mathbf{s}_{<t}^T) \leq I(\mathcal{G}; \mathbf{s}^G). \quad (15)$$

□

4 Methodology

4.1 Overview

Motivated by Lemma 3.1, we propose a reconstructive graph instruction tuning pipeline, called RGLM, which explicitly optimizes the lower bound of $I(\mathcal{G}; \mathbf{s}^G)$ through extra reconstructive graph tasks. As shown in Figure 3(a), a graph $\mathcal{G}$ is first serialized into a token sequence using the neighbor detail template $\mathcal{F}$. These tokens are aligned with the text space via a ϕ -parameterized projector, and then used as prefix tokens to query the θ -parameterized LLM. Our RGLM aims to reconstruct graph information from the LLM-generated graph token representation $\mathbf{s}^G$. Since $\mathbf{s}^G$ may contain multiple tokens for the same node (e.g., node A appears four times in Figure 2), we apply mean aggregation over these tokens to obtain the reconstruction token representation $\mathbf{H} = \{\mathbf{h}_1, \dots, \mathbf{h}_{|\mathcal{V}|}\}$:

$$\mathbf{h}_v = \frac{1}{|\Gamma(v)|} \sum_{i \in \Gamma(v)} \mathbf{s}_i^G, \quad \forall v \in \mathcal{V}, \quad (16)$$

where $\Gamma(v) \subseteq \{1, \dots, N\}$ denotes the index set of tokens associated with node v in $\mathbf{s}^G$. Here, to reduce computational cost and avoid

catastrophic forgetting [19], we adopt LoRA [14] for fine-tuning the LLM. The training objective comprises two components:

$$\mathcal{L}_{\text{RGLM}} = \mathcal{L}_{\text{text}} + \mathcal{L}_{\text{graph}}, \quad (17)$$

where $\mathcal{L}_{\text{text}}$ is the textual autoregressive loss defined in Equation (2), and $\mathcal{L}_{\text{graph}}$ is an extra reconstruction term on graph data. The $\mathcal{L}_{\text{graph}}$ is defined using a customized metric $\mathcal{M}$ that measures the discrepancy between the reconstruction tokens $\mathbf{H}$ and a predefined target derived from the graph $\mathcal{G}$. Formally, this loss is expressed as:

$$\mathcal{L}_{\text{graph}}(\Theta = \{\phi, \theta, \pi\}, \mathcal{G}, \mathbf{x}) = \mathcal{M}(\mathcal{J}_\pi(\mathbf{H}), \mathcal{R}(\mathcal{G})), \quad (18)$$

where $\mathcal{J}_\pi$ is a π -parameterized post-projection function, which maps $\mathbf{H}$ to be consistent with the predefined target. The function $\mathcal{R}$ constructs the reconstruction target from $\mathcal{G}$, which may include any observable graph signals. Evidently, the choice of $\mathcal{R}$ and $\mathcal{M}$ yields different variants. In the following, we explore three variants with alignment guarantees from two complementary perspectives. In the input space, we consider a variant that directly reconstructs the raw node features and topology. In the latent space of pre-trained GNNs, we examine two semantic variants that reconstruct latent node representations: one uses a similarity-based objective while the other employs a denoising strategy.

4.2 Raw Graph Reconstruction in Input Space

In the input space, we introduce RGLM-Decoder, which reconstructs the raw graph information from $\mathbf{H}$. As shown in Figure 3(b), we train a feature decoder d_f and a structure decoder d_s to reconstruct the node features and the adjacency, respectively. In this setting, $\mathcal{J}_\pi$ is instantiated by d_f and d_s , while $\mathcal{R}$ consists of a pre-trained language model for text encoding and an adjacency matrix extractor. The metric $\mathcal{M}$ is defined as the mean squared error (MSE) for feature reconstruction and the binary cross entropy (BCE) for structure reconstruction. The objective function is formulated as:

$$\mathcal{L}_{\text{graph}} = \lambda_f \cdot \mathcal{L}_{\text{feat}} + \lambda_s \cdot \mathcal{L}_{\text{topo}}, \quad (19)$$

$$\mathcal{L}_{\text{feat}} = \frac{1}{|\mathcal{V}|} \sum_{i=1}^{|\mathcal{V}|} \|d_f(\mathbf{h}_i) - \mathbf{z}_i\|_2^2, \quad (20)$$

$$\begin{aligned} \mathcal{L}_{\text{topo}} &= -\frac{1}{|\mathcal{E}|} \sum_{(i,j) \in \mathcal{E}} \log \sigma(d_s(\mathbf{h}_i, \mathbf{h}_j)) \\ &\quad - \frac{1}{|\hat{\mathcal{E}}|} \sum_{(i,j') \in \hat{\mathcal{E}}} \log (1 - \sigma(d_s(\mathbf{h}_i, \mathbf{h}_{j'}))), \end{aligned} \quad (21)$$

where $\mathcal{E}$ and $\hat{\mathcal{E}}$ represent sets of existing and non-existing connections in $\mathcal{G}$, respectively, and $\mathbf{z}$ denotes the node feature. $\sigma(\cdot)$ is the sigmoid function, λ_f and λ_s balance the feature and structural reconstruction losses, respectively.

PROPOSITION 4.1 (LOWER BOUND VIA RGLM-DECODER). *Given a graph $\mathcal{G} = (\mathbf{Z}, \mathbf{A})$, where $\mathbf{Z}$ denotes the node feature matrix and $\mathbf{A}$ the adjacency matrix, minimizing the reconstruction losses $\mathcal{L}_{\text{feat}}$ and $\mathcal{L}_{\text{topo}}$ provides a variational lower bound of $I(\mathcal{G}; \mathbf{s}^G)$:*

$$I(\mathcal{G}; \mathbf{s}^G) \geq \mathcal{H}(\mathbf{Z}) + \mathcal{H}(\mathbf{A}) - \frac{1}{\lambda_f} \mathcal{L}_{\text{feat}} - \frac{1}{\lambda_s} \mathcal{L}_{\text{topo}}. \quad (22)$$

The proof is given in Appendix B.1. Proposition 4.1 indicates minimizing the raw graph reconstruction loss maximizes a computable

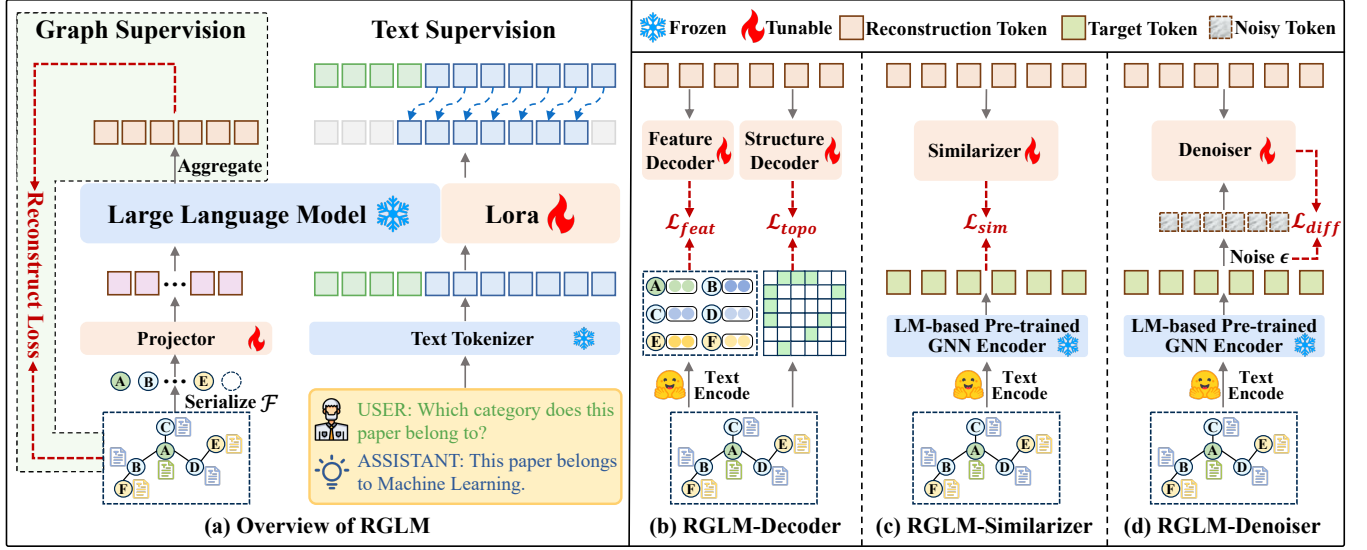


Figure 3: (a) Overview of RGLM. Given an input TAG, RGLM aims to reconstruct the graph information from the LLM’s graph token outputs, explicitly incorporating graph supervision to constrain the alignment process. (b) Decoder variant directly reconstructs the raw node features and topology in the input space. (c) Similarizer and (d) Denoiser variants reconstruct latent node representations of pre-trained GNNs via the cosine-similarity and denoising strategy, respectively.

lower bound of $\mathcal{I}(\mathcal{G}; s^G)$, which directly improves the upper bound of optimization objective C-GTMI $\mathcal{I}(x_t; s^G | s_{<t}^T)$.

4.3 Node Representation Reconstruction in Latent Space

Unlike reconstructing the raw graph information in the input space, this subsection focuses on reconstructing node representation in the latent space of pre-trained GNNs. Specifically, we first pre-train an LM-based GNN encoder g to obtain fine-grained node representations, and then train a lightweight module, Similarizer or Denoiser, to reconstruct node-level semantics from the $\mathbf{H}$.

LM-based Pre-trained GNN Encoder. To obtain fine-grained node representations that effectively capture discrete graph structures and multi-level feature information (i.e., node-, edge-, and graph-level), we pretrain an LM-based GNN encoder g within an autoencoder framework [45, 55]. Specifically, given a full TAG $\mathcal{G}_{full} = (\mathcal{V}_{full}, \mathcal{E}_{full}, \mathcal{S}_{full})$, we first encode the textual attributes $\mathcal{S}_{full}$ using Sentence-BERT [28] to extract the initial node features $\mathbf{Z}_{full}$. Following [27, 55], to further enhance structural awareness, we incorporate learnable edge embeddings by treating absent edges as a special edge type, while introducing LapPE and RWSE positional encodings. We adopt the combination of DeeperGCN [21] and GAT [33] as the backbone of the encoder. Since the latent space is already powerful, the decoder can be a relatively simple linear layer. In the training stage, we randomly mask node labels of the training set for prediction, which can be regarded as a special case of graph reconstruction. The objective function is defined as follows:

$$\mathcal{L}_{pretrain} = \mathcal{L}_{mask} + \mathcal{L}_{reg}, \quad (23)$$

where $\mathcal{L}_{mask}$ denotes the loss for masked label prediction, and $\mathcal{L}_{reg}$ is a regularization term, implemented as a KL-penalty to push the

learned latent representations towards a standard normal distribution [45]. Once the encoder g is well trained, it is used to encode each subgraph $\mathcal{G}$ and produce the target token representations $\mathbf{E}$, where g corresponds to the reconstruction function $\mathcal{R}$ in Equation (18).

RGLM-Similarizer. As shown in Figure 3(c), RGLM-Similarizer trains a lightweight multilayer perceptron s , referred to as the Similarizer, to directly regress the reconstruction token representations $\mathbf{H}$ onto the fine-grained target token representations $\mathbf{E}$. In this setting, $\mathcal{M}$ is defined as the cosine similarity loss, and $\mathcal{J}$ corresponds to the s . The objective function is formulated as:

$$\mathcal{L}_{graph} = \lambda_l \cdot \mathcal{L}_{sim}, \quad \mathcal{L}_{sim} = 1 - \frac{\langle s(\mathbf{H}), \mathbf{E} \rangle}{\|s(\mathbf{H})\|_2 \|\mathbf{E}\|_2}, \quad (24)$$

where λ_l is a scaling factor that balances the reconstruction losses.

PROPOSITION 4.2 (LOWER BOUND VIA RGLM-SIMILARIZER). Given the latent node representations $\mathbf{E}$ of a graph $\mathcal{G}$ obtained from a pre-trained GNN g , minimizing the cosine similarity loss $\mathcal{L}_{sim}$ provides a variational lower bound of $\mathcal{I}(\mathcal{G}; s^G)$:

$$\mathcal{I}(\mathcal{G}; s^G) \geq \mathcal{H}(\mathbf{E}) - (\kappa \cdot \lambda_l \cdot \mathcal{L}_{sim} + C). \quad (25)$$

where $\kappa > 0$ and C is a constant independent of s^G . The proof is given in Appendix B.2.

RGLM-Denoiser. RGLM-Similarizer is a result-oriented reconstruction, which directly regresses $\mathbf{H}$ onto $\mathbf{E}$. In contrast, RGLM-Denoiser is a process-oriented reconstruction by leveraging a denoising objective. As illustrated in Figure 3(d), RGLM-Denoiser trains a denoiser f that conditions on the reconstruction tokens $\mathbf{H}$ to recover clean latent tokens $\mathbf{E}$ from their noisy tokens $\mathbf{E}_t$. The denoiser f is implemented as a stack of transformer blocks, each equipped with three additional projection layers for the conditions $\mathbf{H}$, the noisy inputs $\mathbf{E}_t$, and the timesteps t , respectively. In this setting, $\mathcal{M}$ is defined as the MSE loss between the predicted and

Table 1: Node classification results under supervised settings. Best are bolded, and runner-up are underlined. “OOM” indicates out-of-memory, exceeding the A800’s 80 GB limit. “Rank” represents the overall ranking on all datasets.

Method	Dataset	Cora		Pubmed		OGBN-Arxiv		Reddit		Rank	
	Metric	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
GNNs	GCN	89.30	87.95	89.48	89.04	73.83	54.97	63.19	62.49	5	6
	GraphSAGE	87.82	86.77	90.54	90.51	74.47	56.41	58.51	58.41	10	8
	GAT	87.08	86.32	88.11	87.60	74.39	57.71	61.78	65.38	9	5
	DIFformer	85.60	84.31	81.60	81.94	58.80	42.44	55.30	55.09	15	14
	UniMP	87.82	86.32	90.34	90.17	70.96	53.24	59.55	59.53	11	10
	NodeFormer	87.30	85.03	89.92	89.63	68.69	50.64	<u>68.06</u>	<u>67.89</u>	7	7
	GraphCL	86.53	84.50	90.39	90.28	OOM	OOM	57.67	57.67	12	12
	GRACE	89.11	88.12	86.89	86.58	72.00	50.79	65.39	65.39	8	9
GTextLLMs	LGD	85.61	84.61	90.77	90.65	72.88	50.06	65.92	62.00	6	11
	GraphText-ICL	82.29	81.16	56.72	52.55	68.67	51.67	56.50	52.23	16	16
GTokenLLMs	GraphText-SFT	84.69	83.77	87.80	87.68	49.88	19.46	62.20	62.07	14	15
	GraphGPT	83.92	82.72	81.76	80.99	71.59	54.58	62.11	62.39	13	13
	LLaGA	88.75	87.87	90.34	90.25	74.61	56.48	66.38	66.38	4	4
	RGLM-Decoder	89.85	89.39	91.15	91.07	75.00	56.92	68.64	68.54	1	1
	RGLM-Similarizer	<u>89.67</u>	88.96	91.61	91.51	75.14	57.74	66.79	66.77	3	2
	RGLM-Denoiser	90.22	<u>89.35</u>	90.95	90.90	<u>75.10</u>	56.84	67.64	67.50	2	3

true noise, and $\mathcal{J}$ corresponds to the f . The training procedure follows a standard diffusion process:

$$\mathcal{L}_{\text{graph}} = \lambda_l \cdot \mathcal{L}_{\text{diff}}, \quad \mathcal{L}_{\text{diff}} = \mathbb{E}_{t, \epsilon} [\|f(\mathbf{E}_t, \mathbf{H}, t) - \epsilon\|_2^2], \quad (26)$$

where λ_l is a scaling factor that balances the reconstruction losses. $\mathbf{E}_t$ is sampled from $\mathcal{N}(\sqrt{1 - \beta_t} \mathbf{E}_{t-1}, \beta_t \mathbf{I})$, and $\mathbf{E}_{t-1}$ can be reparameterized directly from $\mathbf{E}$ by letting $\tilde{\alpha}_t = \prod_{i=1}^t (1 - \beta_i)$:

$$\mathbf{E}_t = \sqrt{\tilde{\alpha}_t} \mathbf{E} + \sqrt{1 - \tilde{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (27)$$

PROPOSITION 4.3 (LOWER BOUND VIA RGLM-DENOISER). *Given the latent node representations $\mathbf{E}$ of a graph $\mathcal{G}$ obtained from a pre-trained GNN g , minimizing the diffusion noise prediction loss $\mathcal{L}_{\text{diff}}$ provides a variational lower bound of $\mathcal{I}(\mathcal{G}; s^G)$:*

$$\mathcal{I}(\mathcal{G}; s^G) \geq \mathcal{H}(\mathbf{E}) - (\lambda_l \cdot \mathcal{L}_{\text{diff}} + C). \quad (28)$$

where C is a constant independent of s^G . Proof see Appendix B.3.

5 Experiment

5.1 Experiment Setup

Datasets. We conduct experiments on four widely-used datasets: Cora [46], Pubmed [46] and OGBN-Arxiv [15] from citation networks, as well as Reddit [16] from a social network. Following [3, 22], we select two widely adopted metrics: accuracy (Acc($\uparrow$)) and macro-F1 (F1($\uparrow$)). More dataset details refer to Appendix C.1.

Baselines. We compare five categories of representative baselines: (i) Traditional GNNs, GCN [18], GAT [33] and GraphSAGE [11]; (ii) Graph transformer models, UniMP [29], DIFformer [39] and NodeFormer [40]; (iii) Pretrain-finetune GNNs, GraphCL [48], GRACE [57] and LGD [55]; (iv) The GTextLLMs, GraphText-ICL [54]; (v) The GTokenLLMs, GraphText-SFT [54], GraphGPT [30] and LLaGA [3]. We adopt Sentence-BERT [28] to extract node features from the raw textual attributes, and all GTokenLLMs use Vicuna-7B-v1.5-16K [6] as the base LLM. More baseline introduction and implementation details refer to Appendix C.2 and Appendix C.3, respectively.

Table 2: Link prediction results under supervised settings. Best are bolded, and runner-up are underlined.

Method	Dataset	Cora		Pubmed		Reddit	
	Metric	Acc	F1	Acc	F1	Acc	F1
Baselines	GCN	84.26	83.91	90.00	89.98	74.38	73.85
	UniMP	71.62	70.92	80.76	80.54	73.09	72.53
	GRACE	70.44	69.19	81.74	81.56	61.68	60.88
	GraphGPT	59.85	59.85	73.60	73.39	56.69	27.19
	LLaGA	82.21	79.66	91.78	91.88	79.73	76.60
Ours	RGLM-Decoder	88.97	87.96	94.84	94.91	81.90	<u>77.97</u>
	RGLM-Similarizer	89.85	88.82	94.67	94.72	81.83	78.10
	RGLM-Denoiser	89.71	88.92	94.67	94.74	81.86	77.93

5.2 Main Results

5.2.1 Performance on Node Classification. We mainly focus on the supervised node classification task. In Table 1, we find: (1) RGLM consistently outperforms all baselines. It highlights RGLM’s emergent ability to tackle complex graph tasks beyond traditional GNNs. Compared to existing GTokenLLMs, RGLM introduces explicit graph supervision, which yields more effective alignment, leading to superior performance. (2) Among the three RGLM variants, Decoder achieves the overall best performance. A possible reason is that reconstructing both the node features and the graph topology in the input space provides more comprehensive supervision for alignment. (3) Although we follow the pretraining framework of LGD for learning node latent representations, the performance of Similarizer/Denoiser is not upper-bounded by it. This is because graph reconstruction serves as a regularization term, aiming to enhance the attainable performance ceiling of the model.

5.2.2 Performance on Link Prediction. We also evaluate RGLM on the supervised link prediction task. Following works [3, 30], we randomly select node pairs from the node classification training set for training and from the test set for evaluation, ensuring the edge-level training sets match the size of the node-level. As shown in Table 2, RGLM consistently outperforms all baselines in the link

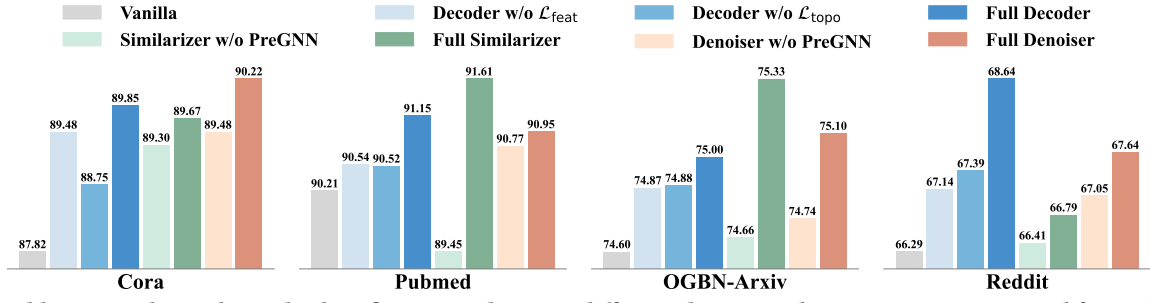


Figure 4: Ablation study on the node classification task across different datasets, where accuracy is reported for each variant.

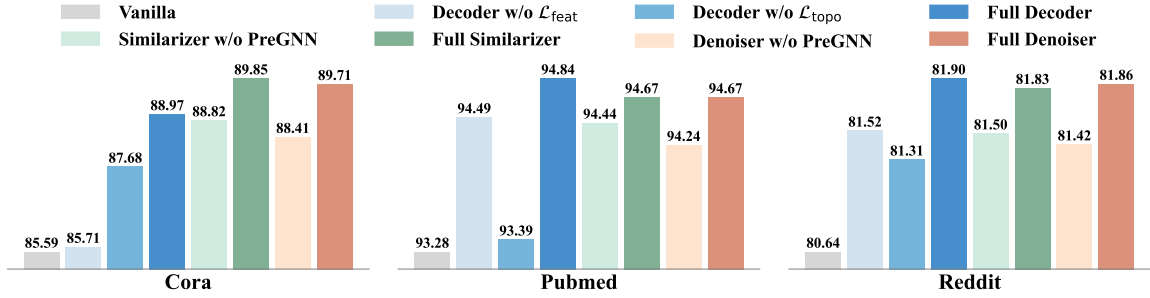


Figure 5: Ablation study on the link prediction task across different datasets, where accuracy is reported for each variant.

Table 3: Node classification results under multi-dataset training. Best are bolded, and runner-up are underlined.

Method	Training set	Cora+Pubmed			
	Test set	Cora		Pubmed	
	Metric	Acc	F1	Acc	F1
Baselines	GCN	85.61	84.16	85.07	84.29
	GraphSAGE	86.35	86.21	89.27	89.12
	GAT	83.03	82.29	86.51	85.99
	UniMP	86.35	85.21	89.20	89.14
	NodeFormer	87.30	85.44	88.44	88.22
	GRACE	88.56	87.55	86.21	85.92
	GraphGPT	81.59	79.83	78.21	77.45
	LLaGA	88.53	87.28	89.60	89.46
Ours	RGLM-Decoder	89.67	88.14	90.37	90.32
	RGLM-Similarizer	88.93	87.49	91.18	91.07
	RGLM-Denoiser	90.04	88.94	90.34	90.36

prediction task. Compared to GraphGPT and LLaGA, which solely leverage text supervision, RGLM explicitly incorporates graph supervision into the alignment process, resulting in an accuracy improvement of 2.1 to 30 points. This highlights the critical role of explicit graph supervision in supporting graph-text alignment.

5.2.3 Multi-dataset Generalization Ability. Typically, training a model on multiple datasets may lead to catastrophic forgetting [19], resulting in degraded performance. Here, we examine the multi-dataset generalization capability of RGLM across multiple datasets. In our study, the models are first trained on node classification tasks using both the Cora and Pubmed, and then evaluated on each dataset. As reported in Table 3, RGLM remains robust when trained across different datasets, achieving superior performance and demonstrating its capability in cross-dataset learning.

5.2.4 Zero-Shot Ability. We investigate the zero-shot capability of RGLM, where the model is trained on a source dataset and

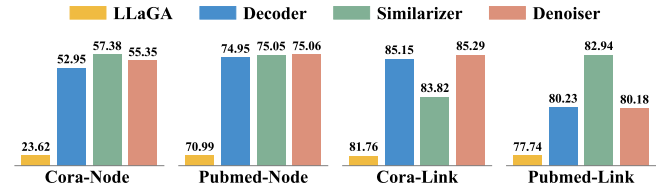


Figure 6: Zero-shot accuracy on node classification and link prediction tasks, where models are trained on the OGBN-Arxiv and evaluated on unseen Cora and Pubmed.

evaluated on a target dataset without any access to the target data during training. Following LLaGA [3], we train the model on the node description and link prediction tasks with OGBN-Arxiv, and evaluate it on node classification and link prediction using Cora and Pubmed, respectively. As shown in Figure 6, the three RGLM variants have robust zero-shot capabilities across both tasks. By introducing the additional graph reconstruction tasks, the knowledge can be more effectively transferred across different scenarios.

5.3 Model Analysis

5.3.1 Ablation Study. We first evaluate the importance of key components through an ablation study. Figures 4 and 5 present the ablation results of the three RGLM variants on node classification and link prediction, respectively. Specifically, “Vanilla” denotes the setting without any reconstruction loss, where LLMs are tuned solely through text supervision. “w/o $\mathcal{L}_{feat}$ ” and “w/o $\mathcal{L}_{topo}$ ” correspond to removing the feature decoder and the structure decoder, respectively. “w/o PreGNN” refers to removing the pre-trained GNN, which is equivalent to reconstructing the input features while ignoring structural information on the graph. Based on the results, we observe: (1) Explicitly incorporating graph supervision into the instruction tuning process consistently improves performance compared with pure text supervision, demonstrating the effectiveness

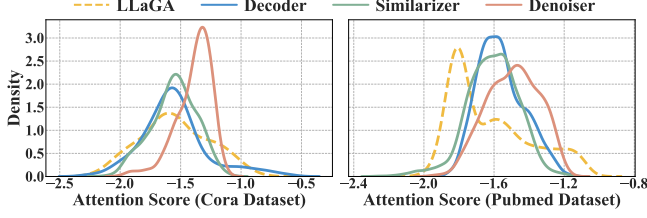


Figure 7: Attention distribution over graph tokens for different methods, where the attention scores are log-scaled.

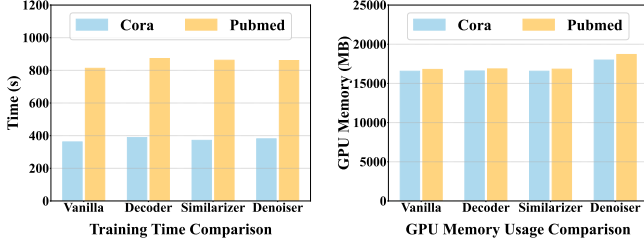


Figure 8: Time complexity (left) and resource cost (right) analysis of different RGLM variants.

of RGLM. (2) For Decoder, introducing both feature reconstruction and structure reconstruction consistently improves performance, with the best results achieved when both are applied together. (3) For Similarizer and Denoiser, even without the pre-trained GNN, RGLM still yields performance gains in most cases, which is because they approximately degenerate into input feature reconstruction.

5.3.2 Attention Analysis. We compute the attention scores of the last token with respect to all graph tokens, excluding the placeholder tokens, on the test set of Cora and Pubmed. Figure 7 illustrates the attention distribution over graph tokens for LLaGA and RGLM. The results reveal that RGLM yields significantly higher attention scores than LLaGA, indicating that incorporating the graph reconstruction objective effectively guides the LLM to focus on input graphs and enhances its ability to comprehend graphs.

5.3.3 Compatibility Analysis. We separately adopt LLaMA3-8B [31] and Vicuna-13B [6] as base LLMs to evaluate RGLM’s compatibility. We report the accuracy of various LLMs under two settings: without the reconstruction loss (“Vanilla”) and with the proposed RGLM. As shown in Figure 9, introducing reconstructive graph loss consistently improves the performance of various LLMs compared with pure text supervision, demonstrating that RGLM is compatible with diverse LLMs and serves as a universal pipeline.

5.3.4 Complexity Analysis. We further evaluate the time and space complexity of RGLM. Since the graph reconstruction loss does not affect the inference stage of the LLM, here we focus on the instruction tuning phase. Figure 8 reports the training time and GPU memory overhead of the three variants on the Cora and Pubmed datasets. The results indicate that the reconstruction process introduces only a negligible increase in training time and resources, while the performance gains clearly outweigh this minor overhead.

5.3.5 Hyper-parameter Analysis. Finally, we conduct sensitivity analysis on the reconstruction loss coefficients: λ_f and λ_s for Decoder, λ_l for Similarizer and Denoiser. As shown in Figure 10 and

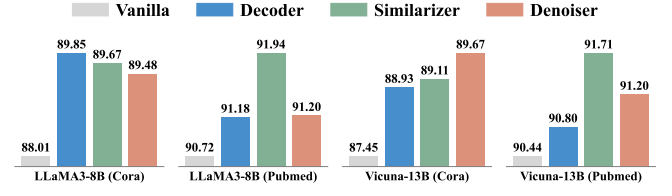


Figure 9: Compatibility analysis of different RGLM variants on base LLMs, where accuracy is reported for each variant.

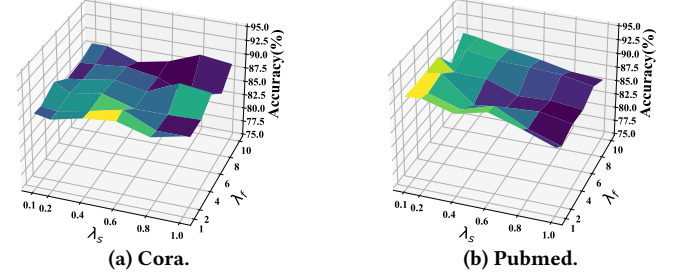


Figure 10: Analysis of the hyper-parameter λ_s and λ_f in the RGLM-Decoder variant.

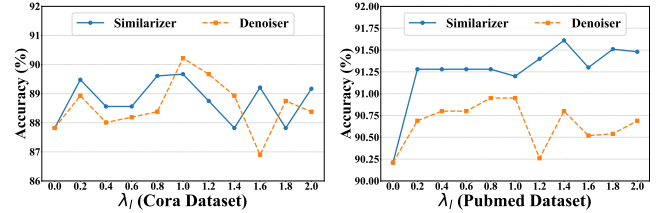


Figure 11: Analysis of the hyper-parameter λ_l in the RGLM-Similarizer and RGLM-Denoiser variants.

Figure 11, RGLM’s performance increases at first and then decreases as the reconstruction loss weight grows. This trend stems from the role of reconstruction objectives as regularization terms: when the weight is too small, the explicit graph supervision becomes insufficient, leading to under-utilization of graph information; when the weight is too large, the reconstruction loss dominates training and interferes with language modeling, degrading the performance.

6 Conclusion

In this work, we theoretically reveal the limitations of existing GTokenLLMs that rely solely on text supervision from language instructions, resulting in implicit graph-text alignment. To overcome it, we prove that the alignment objective is upper-bounded by the mutual information between the input graphs and their hidden representations in the LLM, and further propose a reconstructive graph instruction tuning pipeline, RGLM, to improve this upper bound. The RGLM is embodied through three distinct variants: Decoder from the input space; Similarizer and Denoiser from the latent space. Extensive experiments validate RGLM’s effectiveness.

Acknowledgments

This work is supported in part by the National Natural Science Foundation of China (No. 62550138, 62192784, 62572064, 62472329, 62322203, 62172052), and the BUPT Excellent Ph.D. Students Foundation (No. CX20251005).

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Jinwen Chen, Hao Miao, Dazhuo Qiu, et al. 2025. Sustainability-Oriented Task Recommendation in Spatial Crowdsourcing. In *ICDE*.
- [3] Runjin Chen, Tong Zhao, Ajay Kumar Jaiswal, et al. 2024. LLaGA: Large Language and Graph Assistant. In *International Conference on Machine Learning*. PMLR.
- [4] Xi Chen, Yan Duan, Rein Houthooft, et al. 2016. Infogan: Interpretable representation learning by information maximizing generative adversarial nets. *Advances in neural information processing systems* 29 (2016).
- [5] Zhikai Chen, Haitao Mao, et al. 2024. Exploring the potential of large language models (llms) in learning on graphs. *ACM SIGKDD Explorations Newsletter* (2024).
- [6] Wei-Lin Chiang, Zhuohan Li, Ziqing Lin, et al. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90% chatgpt quality. See <https://vicuna.lmsys.org> (accessed 14 April 2023) 2, 3 (2023), 6.
- [7] Jacob Devlin, Ming-Wei Chang, et al. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *North American Chapter of the Association for Computational Linguistics*.
- [8] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. Talk like a Graph: Encoding Graphs for Large Language Models. In *The Twelfth International Conference on Learning Representations*.
- [9] Matthias Fey and Jan Eric Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428* (2019).
- [10] Jiayan Guo, Lun Du, Hengyu Liu, et al. 2023. Gpt4graph: Can large language models understand graph structured data? an empirical evaluation and benchmarking. *arXiv preprint arXiv:2305.15066* (2023).
- [11] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* (2017).
- [12] Yufei He, Yuan Sui, Xiaoxin He, et al. 2025. Unigraph: Learning a unified cross-domain foundation model for text-attributed graphs. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*.
- [13] R Devon Hjelm, Alex Fedorov, Samuel Lavoie-Marchildon, et al. 2019. Learning deep representations by mutual information estimation and maximization. In *International Conference on Learning Representations*.
- [14] Edward J Hu, Yelong Shen, Phillip Wallis, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [15] Weihua Hu, Matthias Fey, Marinka Zitnik, et al. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* (2020).
- [16] Xuanwen Huang, Kaiqiao Han, Yang Yang, et al. 2024. Can gnn be good adapter for llms?. In *Proceedings of the ACM Web Conference 2024*.
- [17] Bowen Jin, Gang Liu, et al. 2024. Large language models on graphs: A comprehensive survey. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [18] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- [19] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, et al. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences* (2017).
- [20] Lecheng Kong, Jiarui Feng, Hao Liu, et al. 2025. GOFA: A Generative One-For-All Model for Joint Graph Language Modeling. In *The Thirteenth International Conference on Learning Representations*.
- [21] Guohao Li, Chenxin Xiong, Ali Thabet, et al. 2020. Deepergcn: All you need to train deeper gcns. *arXiv preprint arXiv:2006.07739* (2020).
- [22] Yuhao Li, Peisong Wang, Xiao Zhu, et al. 2024. Glbench: A comprehensive benchmark for graph with large language models. *Advances in Neural Information Processing Systems* 37 (2024), 42349–42368.
- [23] Jiawei Liu, Cheng Yang, Zhiyuan Lu, et al. 2025. Graph foundation models: Concepts, opportunities and challenges. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).
- [24] Yinghui Liu, Hao Miao, Guojing Shen, et al. 2025. SPOT-Trip: Dual-Preference Driven Out-of-Town Trip Recommendation. In *NeurIPS*.
- [25] Renqiang Luo, Huafei Huang, Ivan Lee, et al. 2025. FairGP: A Scalable and Fair Graph Transformer Using Graph Partitioning. In *Proceedings of the 39th Annual AAAI Conference on Artificial Intelligence*.
- [26] Renqiang Luo, Huafei Huang, Shuo Yu, et al. 2024. FairGT: A Fairness-aware Graph Transformer. In *Proceedings of the 32nd International Joint Conference on Artificial Intelligence*.
- [27] Ladislav Rampásek, Michael Galkin, et al. 2022. Recipe for a general, powerful, scalable graph transformer. *Advances in Neural Information Processing Systems* (2022).
- [28] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.
- [29] Yunsheng Shi, Zhengjie Huang, Shikun Feng, et al. 2021. Masked Label Prediction: Unified Message Passing Model for Semi-Supervised Classification. International Joint Conferences on Artificial Intelligence Organization.
- [30] Jiabin Tang, Yuhao Yang, Wei Wei, et al. 2024. Graphgpt: Graph instruction tuning for large language models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*.
- [31] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [32] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. 2017. Attention is all you need. *Advances in neural information processing systems* (2017).
- [33] Petar Veličković, Guillem Cucurull, Arantxa Casanova, et al. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- [34] Duo Wang, Yuan Zuo, Fengzhi Li, et al. 2024. Llm as zero-shot graph learners: Alignment of gnn representations with llm token embeddings. *Advances in Neural Information Processing Systems* 37 (2024), 5950–5973.
- [35] Heng Wang, Shangbin Feng, Tianxing He, et al. 2023. Can language models solve graph problems in natural language? *Advances in Neural Information Processing Systems* 36 (2023), 30840–30861.
- [36] Haoyu Peter Wang, Shikun Liu, Rongzhe Wei, et al. 2025. Generalization Principles for Inference over Text-Attributed Graphs with Large Language Models. In *Forty-second International Conference on Machine Learning*.
- [37] Jianing Wang, Junda Wu, Yupeng Hou, et al. 2024. Instructgraph: Boosting large language models via graph-centric instruction tuning and preference alignment. *arXiv preprint arXiv:2402.08785* (2024).
- [38] Fei Wu, Xiao-Yuan Jing, Pengfei Wei, et al. 2022. Semi-supervised multi-view graph convolutional networks with application to webpage classification. *Information Sciences* 591 (2022), 142–154.
- [39] Qitian Wu, Chenxiao Yang, Wentao Zhao, et al. 2023. Difformer: Scalable (graph) transformers induced by energy constrained diffusion. *arXiv preprint arXiv:2301.09474* (2023).
- [40] Qitian Wu, Wentao Zhao, Zenan Li, et al. 2022. Nodeformer: A scalable graph structure learning transformer for node classification. *Advances in Neural Information Processing Systems* (2022).
- [41] Zonghan Wu, Shirui Pan, Fengwen Chen, et al. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* (2020).
- [42] Bo Yan, Yang Cao, Haoyu Wang, et al. 2024. Federated heterogeneous graph neural network for privacy-preserving recommendation. In *Proceedings of the ACM Web Conference 2024*.
- [43] Bo Yan, Sihao He, Cheng Yang, et al. 2025. Federated graph condensation with information bottleneck principles. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [44] Jinyu Yang, Ruijia Wang, et al. 2025. Harnessing Language Model for Cross-Heterogeneity Graph Knowledge Transfer. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [45] Ling Yang, Ye Tian, Minkai Xu, et al. 2024. VQGraph: Rethinking Graph Representation Space for Bridging GNNs and MLPs. In *The Twelfth International Conference on Learning Representations*.
- [46] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*.
- [47] Ruosong Ye, Caiqi Zhang, Runhui Wang, et al. 2024. Language is All a Graph Needs. In *EACL (Findings)*.
- [48] Yuning You, Tianlong Chen, Yongduo Sui, et al. 2020. Graph contrastive learning with augmentations. *Advances in neural information processing systems* (2020).
- [49] Chaochao Yuan, Kangfei Zhao, Ercan Engin Kuruoglu, et al. 2025. A survey of graph transformers: Architectures, theories and applications. *arXiv preprint arXiv:2502.16533* (2025).
- [50] Mengmei Zhang, Mingwei Sun, Peng Wang, et al. 2024. Graphtranslator: Aligning graph model to large language model for open-ended tasks. In *Proceedings of the ACM Web Conference 2024*. 1003–1014.
- [51] Zhongjian Zhang, Xiao Wang, et al. 2025. Can large language models improve the adversarial robustness of graph neural networks?. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*.
- [52] Zhongjian Zhang, Mengmei Zhang, et al. 2024. Endowing pre-trained graph models with provable fairness. In *Proceedings of the ACM Web Conference 2024*.
- [53] Zhongjian Zhang, Mengmei Zhang, Xiao Wang, et al. 2025. Rethinking Byzantine Robustness in Federated Recommendation from Sparse Aggregation Perspective. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [54] Jianan Zhao, Le Zhuo, Yikang Shen, et al. 2024. GraphText: Graph Reasoning in Text Space. In *Adaptive Foundation Models: Evolving AI for Personalized and Efficient Learning*.
- [55] Cai Zhou, Xiyuan Wang, and Muhan Zhang. 2024. Unifying generation and prediction on graphs with latent graph diffusion. *Advances in Neural Information Processing Systems* (2024).
- [56] Huachi Zhou, Jiahe Du, et al. 2025. Each graph is a new language: Graph Learning with LLMs. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics.
- [57] Yanqiao Zhu, Yichen Xu, et al. 2020. Deep Graph Contrastive Representation Learning. In *ICML Workshop on Graph Representation Learning and Beyond*.

A Related Work

A.1 Learning on Graphs

Graph data are ubiquitous in the real world, including social networks [16, 52], webpage networks [38], citation networks [43, 46], and e-commerce networks [2, 24, 42, 53]. However, due to their inherent irregularity and complex connectivity patterns, effectively modeling graph structure is a fundamental challenge. Graph Neural Networks (GNNs) have emerged as powerful tools to address this issue by leveraging the message-passing mechanism to aggregate information from local neighborhoods [41]. Representative models such as GCN [18], GAT [33] and GraphSAGE [11] have shown notable success in node classification and link prediction tasks. Concurrently, there is a growing interest in adapting transformer-based architectures [32] to graph domains [25, 26, 39, 40, 49]. These models aim to capture global dependencies and richer structural semantics, providing a complementary perspective to traditional local aggregation schemes. Despite these successes, a critical limitation of existing GNNs lies in their poor task generalization. Most existing models are trained in a task-specific and dataset-dependent manner, which hampers their performance when transferred to different distributions or tasks.

A.2 Large Language Models for Graphs

Recently, the remarkable success of large language models (LLMs) [1, 6, 31] has motivated researchers to adapt them to the graph learning domain, with the ultimate goal of developing a graph foundation model that generalizes across diverse scenarios [23, 44]. Typically, LLMs are employed as predictors to accomplish various graph tasks, attracting considerable attention [17]. A key challenge lies in aligning graph data with the natural language embedding space, so that LLMs can better comprehend graphs. To address this challenge, existing efforts mainly fall into Graph-Textualizing LLMs (GTextLLMs) and Graph-Tokenizing LLMs (GTokenLLMs).

GTextLLMs [5, 8, 10, 35, 37, 54] utilize natural language to describe graphs via manually crafted prompt templates, directly aligning them with text. For instance, NLGraph [35] represents graphs as node and edge lists embedded in textual templates. GraphText [54] converts graphs into tree-based textual formats using syntactic graph parsing. However, such strategies often suffer from redundancy and imprecise characterization [3, 5, 30]. In contrast, GTokenLLMs [3, 20, 30, 34, 50] encode complex structures and lengthy texts into a graph token sequence, and treat them as prefix tokens for querying LLMs. To align graph tokens with the text embedding space, they typically construct various language instructions related to graph tasks, and learn a projector to facilitate modalities alignment. For example, GraphGPT [30] builds language instructions around the graph matching task, while LLaGA [3] builds them by node classification, link prediction, and node description tasks. Despite these successes, their alignment process follows an LLM-centric design that relies solely on text supervision, resulting in only implicit graph-text alignment and thus limiting performance.

B Proofs

B.1 Proof of Proposition 4.1

PROOF. Since $\mathbf{H}$ is a deterministic function of $\mathbf{s}^G$ (i.e., mean pooling), the data processing inequality ensures that $I(\mathcal{G}; \mathbf{s}^G) \geq$

$I(\mathcal{G}; \mathbf{H})$. By decoding $\mathbf{H}$ back into node features $\mathbf{Z}$ and adjacency matrix $\mathbf{A}$, where $\mathbf{A}_{ij} = 1$ if nodes v_i and v_j are connected, otherwise $\mathbf{A}_{ij} = 0$, the reconstruction loss can be expressed as:

$$\mathcal{L}_{\text{graph}} = \lambda_f \mathbb{E}[-\log q_f(\mathbf{Z}|\mathbf{H})] + \lambda_s \mathbb{E}[-\log q_s(\mathbf{A}|\mathbf{H})]. \quad (29)$$

Assume $q_f(\mathbf{Z}|\mathbf{H}) = \mathcal{N}(\mu_f(\mathbf{H}), \sigma^2 \mathbf{I})$, which corresponds to MSE loss over features; and $q_s(\mathbf{A}|\mathbf{H}) = \prod_{i < j} \text{Bernoulli}(\pi_s(\mathbf{h}_i, \mathbf{h}_j))$, which corresponds to BCE loss over edges. By the Barber–Agakov identity, for any conditional model q we have:

$$\begin{aligned} \mathbb{E}[-\log q_f(\mathbf{Z}|\mathbf{H})] &= \mathcal{H}(\mathbf{Z}|\mathbf{H}) + \mathbb{E}[\text{KL}(p(\mathbf{Z}|\cdot) \| q_f(\mathbf{Z}|\cdot))] \\ &\geq \mathcal{H}(\mathbf{Z}|\mathbf{H}), \end{aligned} \quad (30)$$

$$\begin{aligned} \mathbb{E}[-\log q_s(\mathbf{A}|\mathbf{H})] &= \mathcal{H}(\mathbf{A}|\mathbf{H}) + \mathbb{E}[\text{KL}(p(\mathbf{A}|\cdot) \| q_s(\mathbf{A}|\cdot))] \\ &\geq \mathcal{H}(\mathbf{A}|\mathbf{H}). \end{aligned} \quad (31)$$

Thus,

$$\mathcal{L}_{\text{graph}} \geq \lambda_f \mathcal{H}(\mathbf{Z}|\mathbf{H}) + \lambda_s \mathcal{H}(\mathbf{A}|\mathbf{H}). \quad (32)$$

Here, we assume the graph as a pair of independent random variables $(\mathbf{Z}, \mathbf{A})$. The mutual information can be decomposed as:

$$\begin{aligned} I(\mathcal{G}; \mathbf{H}) &= I(\mathbf{Z}; \mathbf{H}) + I(\mathbf{A}; \mathbf{H}) \\ &= \mathcal{H}(\mathbf{Z}) - \mathcal{H}(\mathbf{Z}|\mathbf{H}) + \mathcal{H}(\mathbf{A}) - \mathcal{H}(\mathbf{A}|\mathbf{H}). \end{aligned} \quad (33)$$

Since $\mathcal{H}(\mathbf{Z})$ and $\mathcal{H}(\mathbf{A})$ are parameter-independent, combining the above inequality and dropping non-negative KL terms yields:

$$I(\mathcal{G}; \mathbf{s}^G) \geq \mathcal{H}(\mathbf{Z}) + \mathcal{H}(\mathbf{A}) - \frac{1}{\lambda_f} \mathcal{L}_{\text{feat}} - \frac{1}{\lambda_s} \mathcal{L}_{\text{topo}}. \quad (34)$$

□

The proof indicates that minimizing the raw graph reconstruction loss maximizes a computable lower bound of $I(\mathcal{G}; \mathbf{s}^G)$, which directly improves the upper bound of optimization objective C-GTMI $I(\mathbf{x}_t; \mathbf{s}^G | \mathbf{s}_{<t}^T)$.

B.2 Proof of Proposition 4.2

PROOF. Given a subgraph $\mathcal{G}$, let $\mathbf{E} = g(\mathcal{G})$ denote the latent node representation matrix extracted by the pre-trained GNN g . The reconstruction tokens $\mathbf{H}$ are mapped to $\mathbf{E}$ by a lightweight MLP $s(\cdot)$, with normalization ensuring $\|\mathbf{E}\| = \|\mathbf{s}(\mathbf{H})\| = 1$. The reconstruction loss is defined as the cosine similarity:

$$\mathcal{L}_{\text{sim}} = \mathbb{E}[1 - \cos(s(\mathbf{H}), \mathbf{E})]. \quad (35)$$

On the unit sphere, we assume that $\mathbf{E}$ conditioned on $\mathbf{H}$ follows a von Mises–Fisher (vMF) distribution:

$$q_s(\mathbf{E}|\mathbf{H}) = \text{vMF}(\mathbf{E}; \mu(\mathbf{H}), \kappa) \propto \exp(\kappa \cdot \lambda_l \cdot \cos(\mu(\mathbf{H}), \mathbf{E})), \quad (36)$$

where $\mu(\mathbf{H}) := \frac{s(\mathbf{H})}{\|s(\mathbf{H})\|}$, $\kappa > 0$, λ_l is a scaling factor. It follows that:

$$-\log q_s(\mathbf{E}|\mathbf{H}) = -\kappa \cdot \lambda_l \cdot \cos(s(\mathbf{H}), \mathbf{E}) + \log C_d(\kappa), \quad (37)$$

where $\log C_d(\kappa)$ is a constant independent of $\mathbf{H}$. Therefore,

$$\begin{aligned} \mathbb{E}[-\log q_s(\mathbf{E}|\mathbf{H})] &= \text{const} + \kappa \cdot \lambda_l \cdot \mathbb{E}[1 - \cos(s(\mathbf{H}), \mathbf{E})] \\ &= \text{const} + \kappa \cdot \lambda_l \cdot \mathcal{L}_{\text{sim}}. \end{aligned} \quad (38)$$

For any conditional model $q_s(\mathbf{E}|\mathbf{H})$, we have:

$$\begin{aligned} \mathbb{E}[-\log q_s(\mathbf{E}|\mathbf{H})] &= \mathcal{H}(\mathbf{E}|\mathbf{H}) + \mathbb{E}[\text{KL}(p(\mathbf{E}|\cdot) \| q_s(\mathbf{E}|\cdot))] \\ &\geq \mathcal{H}(\mathbf{E}|\mathbf{H}). \end{aligned} \quad (39)$$

Thus, we obtain:

$$\mathcal{L}_{\text{sim}} \geq \frac{1}{\kappa \cdot \lambda_l} \mathcal{H}(\mathbf{E}|\mathbf{H}) - \text{const} \Rightarrow \mathcal{H}(\mathbf{E}|\mathbf{H}) \leq \kappa \cdot \lambda_l \cdot \mathcal{L}_{\text{sim}} + \text{const}. \quad (40)$$

Since $\mathcal{I}(\mathbf{E}; \mathbf{H}) = \mathcal{H}(\mathbf{E}) - \mathcal{H}(\mathbf{E}|\mathbf{H})$, we obtain:

$$\mathcal{I}(\mathbf{E}; \mathbf{H}) \geq \mathcal{H}(\mathbf{E}) - (\kappa \cdot \lambda_l \cdot \mathcal{L}_{\text{sim}} + \text{const}). \quad (41)$$

Moreover, g is frozen so that $\mathbf{E} = g(\mathcal{G})$ is a deterministic function of $\mathcal{G}$, and $\mathbf{H}$ is a deterministic function of $\mathbf{s}^G$, thus we have:

$$\mathcal{I}(\mathcal{G}; \mathbf{s}^G) \geq \mathcal{I}(\mathcal{G}; \mathbf{H}) \geq \mathcal{I}(\mathbf{E}; \mathbf{H}) \geq \mathcal{H}(\mathbf{E}) - (\kappa \cdot \lambda_l \cdot \mathcal{L}_{\text{sim}} + \text{const}). \quad (42)$$

□

The proof indicates that minimizing the similarity loss between $\mathbf{H}$ and $\mathbf{E}$ maximizes a computable lower bound of $\mathcal{I}(\mathcal{G}; \mathbf{s}^G)$, which in turn improves the attainable upper bound of C-GTMI $\mathcal{I}(\mathbf{x}_t; \mathbf{s}^G | \mathbf{s}_{<t}^T)$.

B.3 Proof of Proposition 4.3

PROOF. Given a subgraph $\mathcal{G}$, let $\mathbf{E} = g(\mathcal{G})$ denote the latent node representation matrix extracted by the pre-trained GNN g . The reconstruction tokens $\mathbf{H}$ are projected into a condition matrix $\mathbf{C} = \Psi(\mathbf{H})$. The objective is to model a conditional distribution $q_f(\mathbf{E}_{t-1} | \mathbf{E}_t, \mathbf{C})$, such that the clean embeddings $\mathbf{E}$ can be recovered from their noisy counterparts $\mathbf{E}_t$ under the condition $\mathbf{C}$. The forward diffusion process is formulated as a Markov chain that gradually perturbs the clean embeddings $\mathbf{E}$ by adding Gaussian noise, thereby constructing noisy embeddings $\mathbf{E}_t$:

$$\begin{aligned} p(\mathbf{E}_t | \mathbf{E}) &= \mathcal{N}(\sqrt{\alpha_t} \mathbf{E}, (1 - \alpha_t) \mathbf{I}), \\ \mathbf{E}_t &= \sqrt{\alpha_t} \mathbf{E} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \end{aligned} \quad (43)$$

where $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ denotes a Gaussian distribution, t indicates discrete timesteps, $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$, and $\beta_t \in (0, 1)$ is a pre-defined variance schedule. A latent diffusion model learns to reverse this progressive noise addition process. Specifically, to iteratively generate clean embeddings $\mathbf{E}$ from pure noise $\mathbf{E}_t$ conditioned on $\mathbf{C}$, the reverse process is parameterized as

$$\begin{aligned} q_f(\mathbf{E}_{t-1} | \mathbf{E}_t, \mathbf{C}) &= \mathcal{N}(\boldsymbol{\mu}_f(\mathbf{E}_t, \mathbf{C}, t), \sigma_t^2 \mathbf{I}), \\ \boldsymbol{\mu}_f &= \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{E}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_f(\mathbf{E}_t; \mathbf{C}, t) \right). \end{aligned} \quad (44)$$

The marginal conditional likelihood is then given by

$$\begin{aligned} \log q_f(\mathbf{E} | \mathbf{H}) &= \log \int q_f(\mathbf{E}_T) \prod_{t=1}^T q_f(\mathbf{E}_{t-1} | \mathbf{E}_t, \mathbf{C}) \\ &\quad \times q_f(\mathbf{E} | \mathbf{E}_1, \mathbf{C}) d\mathbf{E}_{1:T}. \end{aligned} \quad (45)$$

By introducing the forward distribution $p(\mathbf{E}_{1:T} | \mathbf{E})$ as a variational approximation, the standard variational lower bound (ELBO) can be derived:

$$\begin{aligned} \log q_f(\mathbf{E} | \mathbf{H}) &\geq -\text{KL}(p(\mathbf{E}_T | \mathbf{E}) \| q_f(\mathbf{E}_T)) \\ &\quad - \sum_{t=2}^T \mathbb{E} [\text{KL}(p(\mathbf{E}_{t-1} | \mathbf{E}_t, \mathbf{E}) \| q_f(\mathbf{E}_{t-1} | \mathbf{E}_t, \mathbf{C}))] \\ &\quad + \mathbb{E} [\log q_f(\mathbf{E} | \mathbf{E}_1, \mathbf{C})]. \end{aligned} \quad (46)$$

Under a variance-preserving schedule and Gaussian parameterization, each KL term can be rewritten as a mean-squared error

(MSE) on the noise prediction, up to a constant independent of the parameters:

$$\begin{aligned} \text{KL}(p(\mathbf{E}_{t-1} | \mathbf{E}_t, \mathbf{E}) \| q_f(\mathbf{E}_{t-1} | \mathbf{E}_t, \mathbf{C})) &= \tilde{\lambda}_t \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \|\boldsymbol{\epsilon}_f(\mathbf{E}_t; \mathbf{C}, t) - \boldsymbol{\epsilon}\|_2^2 \\ &\quad + \text{const}_t, \end{aligned} \quad (47)$$

where $\tilde{\lambda}_t > 0$ depends on the noise schedule. Substituting back into the ELBO and aggregating constants yields the diffusion loss, which is equivalent to the negative ELBO up to a constant:

$$\mathcal{L}_{\text{diff}} = \sum_{t=1}^T \lambda_t \mathbb{E}_{(\mathbf{E}, \mathbf{H}), \boldsymbol{\epsilon}} \|\boldsymbol{\epsilon}_f(\sqrt{\alpha_t} \mathbf{E} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}; \mathbf{C}, t) - \boldsymbol{\epsilon}\|_2^2, \quad (48)$$

where $\lambda_t > 0$ absorbs the constants into the weights. This equivalence leads to the upper bound on the negative log-likelihood:

$$\mathbb{E}[-\log q_f(\mathbf{E} | \mathbf{H})] \leq \text{const} + \lambda_l \cdot \mathcal{L}_{\text{diff}}, \quad (49)$$

where λ_l is a scaling factor. Moreover, for any conditional model $q_f(\mathbf{E} | \mathbf{H})$ we have:

$$\begin{aligned} \mathbb{E}[-\log q_f(\mathbf{E} | \mathbf{H})] &= \mathcal{H}(\mathbf{E} | \mathbf{H}) + \mathbb{E}[\text{KL}(p(\mathbf{E} | \mathbf{H}) \| q_f(\mathbf{E} | \mathbf{H}))] \\ &\geq \mathcal{H}(\mathbf{E} | \mathbf{H}). \end{aligned} \quad (50)$$

Discarding the non-negative KL term yields the following upper bound on the conditional entropy:

$$\mathcal{H}(\mathbf{E} | \mathbf{H}) \leq \text{const} + \lambda_l \cdot \mathcal{L}_{\text{diff}}. \quad (51)$$

By the definition $\mathcal{I}(\mathbf{E}; \mathbf{H}) = \mathcal{H}(\mathbf{E}) - \mathcal{H}(\mathbf{E} | \mathbf{H})$, we obtain:

$$\mathcal{I}(\mathbf{E}; \mathbf{H}) \geq \mathcal{H}(\mathbf{E}) - (\text{const} + \lambda_l \cdot \mathcal{L}_{\text{diff}}). \quad (52)$$

Finally, since $\mathbf{E} = g(\mathcal{G})$ is a deterministic function of $\mathcal{G}$ and $\mathbf{H}$ is deterministically obtained from $\mathbf{s}^G$ by mean pooling, we have:

$$\mathcal{I}(\mathcal{G}; \mathbf{s}^G) \geq \mathcal{I}(\mathbf{E}; \mathbf{s}^G) \geq \mathcal{I}(\mathbf{E}; \mathbf{H}) \geq \mathcal{H}(\mathbf{E}) - (\text{const} + \lambda_l \cdot \mathcal{L}_{\text{diff}}). \quad (53)$$

□

The proof indicates that minimizing the diffusion noise prediction loss $\mathcal{L}_{\text{diff}}$ maximizes a computable lower bound of $\mathcal{I}(\mathcal{G}; \mathbf{s}^G)$, which in turn improves the attainable upper bound of C-GTMI $\mathcal{I}(\mathbf{x}_t; \mathbf{s}^G | \mathbf{s}_{<t}^T)$.

C Experiment Details

C.1 Dataset Descriptions

We conduct experiments on four widely-used datasets: Cora [46], Pubmed [46] and OGBN-Arxiv [15] from citation networks, as well as Reddit [16] from a social network. The dataset statistics are in Table 4, and the detailed descriptions are listed as follows:

- **Cora** is a citation network in which each node denotes a research paper in the field of computer science, and the associated raw text features include the title and abstract of the paper. Edges represent citation relationships between papers. Each node is assigned a label indicating the paper's category.
- **Pubmed** is a citation network consisting of research papers in the biomedical domain, where each node corresponds to a paper and each edge denotes a citation relationship between two papers.
- **OGBN-Arxiv** is a citation network comprising research papers and their citation relationships, collected from the arXiv repository. Each node represents a paper, and each edge indicates a citation link between two papers.

Table 4: Statistics of experiment datasets.

Dataset	# Nodes	# Edges	# Class	Splitting	Domain
Cora	2,708	5,429	7	6:2:2	citation
Pubmed	19,717	44,338	3	6:2:2	citation
OGBN-Arxiv	169,343	1,166,243	40	6:2:3	citation
Reddit	33,434	198,448	2	1:1:8	social network

• **Reddit** is a social network in which each node represents a user, and edges indicate whether two users have interacted by replying to each other. The raw text associated with each node comprises the content from the user’s previously published subreddit posts, limited to the most recent three entries. Each node is labeled to indicate whether the user is categorized as popular or normal.

C.2 Baseline Descriptions

The traditional GNN models include:

- **GCN** [18] is a popular graph convolutional network based on spectral theory.
- **GAT** [33] is composed of multiple attention layers, which can learn different weights for different neighborhood nodes.
- **GraphSAGE** [11] inductively learns node embeddings by utilizing ego-node features along with a fixed number of neighboring node features.

The transformer-based graph models include:

- **UniMP** [29] employs a Graph Transformer network to integrate both feature and label propagation during training and inference.
- **NodeFormer** [40] filters out dissimilar neighbors using a fixed threshold and aggregates multi-view neighborhood information to address graph inconsistency.
- **DIFFormer** [39] introduces an energy-constrained, diffusion-based graph transformer designed to encode a batch of instances into evolutionary states by incorporating contextual information from other instances.

The pretrain-finetune GNNs include:

- **GraphCL** [48] employs diverse graph augmentation techniques to generate multiple views of a graph and learns node representations by maximizing feature consistency across these views.
- **GRACE** [57] proposes to augment the graph structure by both random edge dropout and random node feature dropping, and uses the corrupted graphs as the contrastive views.
- **LGD** [55] LGD pretrains an autoencoder to embed graphs into a powerful latent space and then trains a diffusion model in this space to unify graph generation and prediction tasks.

The GTextLLMs include:

- **GraphText-ICL** [54] is the in-context learning variant of GraphText, which eliminates the tuning process. GraphText decouples depth and scope by encapsulating node attributes and relationships in the graph syntax tree and processing it with an LLM.

The GTokenLLMs include:

- **GraphText-SFT** [54] is a supervised fine-tuning variant of GraphText, which employs an MLP projector to map node continuous features into the textual input space for querying LLMs.

- **GraphGPT** [30] encodes graph data using a pre-trained GNN and introduces a dual-stage instruction tuning to align graph representations with natural language.
- **LLaGA** [3] designs two graph description templates to serialize graph data and instruction-tuning LLMs with three different tasks, including node description, node classification, and link prediction tasks.

C.3 Implementation Details

First, RGLM aims to explicitly incorporate graph supervision into the tuning process, rather than designing a new instruction dataset. Therefore, we utilize the instruction datasets from LLaGA [3] to train the model. In the tuning stage, we train our model for one epoch with the AdamW optimizer, set the batch size to 4 per device, and set the projector learning rate to $2e-3$. Following LLaGA [3], to compensate for the limited data size, we replicate the training samples from the smallest dataset, Cora and Reddit, three times. The warmup ratio is set to $3e-2$, and the maximum input length of LLMs is 4096. For the Neighbor Detail Template, we sample two-hop neighbors around each node, setting the sample size to 10 for each hop. We adopt LoRA [14] for efficient fine-tuning, with the learning rate, dropout rate, rank, and α set to $5e-4$, 0.1, 8, and 32, respectively. In the inference stage, we set the temperature of the LLM to 0.001 to ensure reproducibility of the results. We uniformly set the learning rate to $5e-4$ for all three RGLM variants and configure the hyperparameters as follows: For RGLM-Decoder, the hyperparameter λ_f is selected from $\{0.1, 0.2, 0.4, 0.6, 0.8, 1.0\}$, and λ_s is selected from $\{1, 2, 4, 6, 8, 10\}$. For RGLM-Similarizer and RGLM-Denoiser, λ_l is chosen from 0.2 to 2.0 with a step size of 0.2. For the LM-based pre-trained GNN encoder, following [27, 55], we use the combination of DeeperGCN [21] and GAT [33] as the backbone, with 7 layers and 96 hidden dimensions. We use LapPE and RWSE. We train the model for 500 epochs with a masked label ratio of 0.8, and a cosine learning rate with 50 warm-up epochs and an initial value of $1e-3$.

To facilitate fair comparisons, we adopt a unified text encoder, Sentence-BERT [28], to extract node features from the raw textual attributes. All GTokenLLMs use Vicuna-7B-v1.5-16K [6] as the base LLM. For these baseline models, we implement them with the source code provided by the authors. Unless otherwise specified, we adopt the default parameter setting in the author’s implementation. For datasets not included in the released baseline code, we tune all baselines using a grid search strategy. For all experiments, we select the optimal hyperparameters on the validation set and apply them to the test set. The implementation of all models uses PyG [9].

C.4 Computing Environment and Resources

The experiments are conducted in a computing environment with the following specifications:

- **OS:** Linux ubuntu 5.15.0-102-generic.
- **CPU:** Intel(R) Xeon(R) Platinum 8358 CPU @ 2.60GHz.
- **GPU:** NVIDIA A800 80GB.